RansomRadar

The Complete Presentation Script & Explainer Guide

Everything you need to explain this project confidently —
in plain language, with the exact words to say.

DEMO SCRIPT · PLAIN-ENGLISH EXPLAINERS · VIVA Q&A · REVIEWER LINES

Minor Project — Third Year, AI & Machine Learning

Cost-Sensitive Early Detection of Crypto-Ransomware
on Filesystem Event Streams Under Throttled Encryption

What's inside

1. The 30-second pitch
   The one paragraph to open with, memorised word for word.
2. The problem, explained like a story
   Why accuracy is the wrong question for ransomware.
3. Every concept in plain English
   Conformal p-values, ECOD, Shapley, intermittent encryption - no jargon.
4. The live demo script, minute by minute
   Exactly what to run, what to point at, and what to say while it runs.
5. The five things that make this stand out
   Your wow factors, and how to deliver each one.
6. What to say to teachers and reviewers
   Lines that earn respect, and the honesty that earns more.
7. The viva Q&A bank
   26 likely questions with answers you can say out loud.
8. If something breaks live
   Recovery lines that make you look calm instead of caught.
9. The cheat sheet
   One page to hold in your hand on the day.

1. The 30-second pitch

This is the first thing out of your mouth. Memorise it. It sets the frame for everything that follows, and it immediately signals that you thought about the *problem*, not just the code.

SAY THIS

"Ransomware detection projects usually report accuracy on a dataset. We think that's the wrong question. If ransomware starts encrypting your files, nobody cares about your F1 score - they care about two things: ***how many files did I lose before the alarm went off****, and* ***how often does this thing cry wolf****. So RansomRadar measures exactly those two numbers and trades them off against each other.*

And because self-evaluation is easy to fake, we built our own attacker to try to defeat us - including the intermittent-encryption trick that real ransomware families use to hide. Then, when we detect an attack, we don't just show a red box. We roll the files back and prove with SHA-256 hashes that they came back byte-for-byte identical."

WHY THIS LANDS

Three moves happen in those 30 seconds: you **reframe the problem** (shows research maturity), you **pre-empt the biggest criticism** of any self-evaluated security project (shows intellectual honesty), and you **promise a visual payoff** (the rollback proof) that makes people want to keep watching.

The single sentence, if you only get one

SAY THIS

"It's an early-warning system that measures files-lost against false-alarms, attacks itself to prove the detection is real, and rolls the damage back with cryptographic proof."

Your project title

Use this on the report cover and the first slide:

Cost-Sensitive Early Detection of Crypto-Ransomware: Trading Files Lost Against False Alarms on Filesystem Event Streams Under Throttled Encryption

DO NOT SAY

Don't call it "An Intelligent ML-Powered Early Warning System". Words like "intelligent" and "powered" are what weak projects use to sound strong. A precise, slightly boring title reads as confident - it says the work speaks for itself. Compare: *"Smart AI-Based Detection System"* versus *"Trading Files Lost Against False Alarms Under Throttled Encryption"*. The second one sounds like a paper.

2. The problem, explained like a story

Use this when you have two or three minutes and someone asks "so what is this about?" It works because it's concrete, and it arrives at your metric naturally instead of announcing it.

SAY THIS

"Imagine ransomware lands on your laptop at 2 a.m. It starts encrypting your files one by one. Every second you don't notice costs you files - permanently.

Now, you could build a detector that's extremely twitchy, so it catches the attack after two files. But then it also screams every time you unzip a download or run a backup, and after the third false alarm you switch it off - and now you have no protection at all.

Or you could build one that's very conservative, never false-alarms, and notices the attack after nine hundred files. Technically it detected it. You've still lost your thesis.

So the real problem isn't 'can you detect ransomware'. It's ***where do you sit on that trade-off****, and can you prove it. That curve is the project."*

Why this beats the usual framing

Typical project says	You say
"We got 98.7% accuracy."	"We lose a median of 4 files, at under one false alarm per benign hour."
"Our model detects ransomware."	"Our model survives an adversary built to evade it."
"We used Isolation Forest."	"We use a threshold with a statistical guarantee, not a number we made up."
"Future scope: add response."	"Here are the files coming back, hash-verified, live."

WHY THIS LANDS

Accuracy is a trap in ransomware and you should say so: most windows of activity are benign, so a detector that always says "normal" scores extremely well on accuracy while being completely useless. Pointing this out unprompted is one of the fastest ways to show you understand your own domain.

3. Every concept in plain English

These are the five ideas someone might quiz you on. Each has a one-line answer, a plain-English explanation, and an analogy you can use if the person looks confused.

3.1 Conformal p-values (your strongest technical card)

One line: Instead of "alert if entropy > 7.0", we say "only 0.08% of known-normal behaviour ever looked this strange".

Plain English: Every detector needs a cut-off. Most projects pick one by trial and error - 7.0, or 0.05, or whatever made the demo work. That number has no meaning outside your laptop. We do it differently. We take a big pile of windows we know are normal, score all of them, and keep those scores as a reference set. When a new window arrives, we ask: *out of all the normal behaviour I've ever seen, what fraction looked at least this weird?* If the answer is 0.0008, that's our p-value.

Why it's powerful: it comes with a mathematical guarantee. If you set your alert level at 1%, you will get at most 1% false alarms on normal traffic - and that holds *without ever having seen an attack*. You're calibrating on normal behaviour alone, which is the only thing you can actually collect in the real world.

> SAY THIS
>
> *"A threshold is a number you guessed. A p-value is a number that comes with a guarantee. We calibrate only on normal behaviour - which is the only data a real deployment can actually collect - and that gives us a false-alarm rate we can set in advance instead of discovering later."*

The analogy: Exam marks. Saying "you scored 71" means nothing on its own. Saying "you're in the top 2% of everyone who ever sat this paper" means everything - because it's relative to a reference population. Our p-value is a percentile against normal file activity.

The formula (write it on the board if asked):

```
p = (1 + number of calibration scores >= new score) / (n_calibration + 1)
```

3.2 ECOD and Isolation Forest (the two detectors)

One line: One looks at each feature's tail, the other carves up the space randomly; we combine them.

Isolation Forest works on a lovely intuition: to isolate a weird point you need very few random cuts, but to isolate a normal point buried in the crowd you need many. So "how few cuts did it take to fence this point off" becomes an anomaly score.

ECOD (Empirical Cumulative Distribution Outlier Detection) asks, for each feature separately: how far into the tail is this value? 400 file writes in ten seconds, when normal never exceeded 30 - that's deep in the tail. It adds those tail rarities up across features. Its big advantage: it's **parameter-free** and it tells you *which feature* made the window suspicious, which is what feeds your explanations.

> SAY THIS
>
> *"We use two detectors because they fail differently. Isolation Forest catches unusual combinations of features; ECOD catches extreme individual values and tells us which feature was responsible. Averaging their normalised scores is more stable than trusting either alone."*

3.3 Intermittent (partial) encryption - why your evader matters

One line: Real ransomware now encrypts only part of each file, specifically to stay invisible to entropy-based detectors.

Plain English: Encrypted data looks like random noise. Random noise has very high entropy - about 7.9 out of 8. So the classic detection trick is: if files suddenly become high-entropy, you're being encrypted. Attackers worked this out. Since around 2022, families like LockFile, BlackCat/ALPHV, Play and Qilin encrypt only a fraction of each file - maybe 15%. The file is still ruined and still unopenable, but its *average* entropy barely moves, so the entropy alarm never fires. It's also faster for them, which is a bonus.

Your contribution: you built this attack yourself, with a dial (`encrypt_fraction`), and you can show the detection curve collapsing as you turn the dial down - while your behavioural features keep working.

> SAY THIS
>
> *"This is the experiment I'd point you at first. I turn the encryption fraction down from 100% to 15%, and you can watch the entropy-only detector go blind while the behavioural features keep catching it. That graph is the argument for why you need more than entropy - and I generated it by attacking myself."*

3.4 Shapley values (the explainability piece)

One line: It splits the alert score fairly among the features, and the parts add up exactly.

Plain English: The detector outputs one number: 94. Which feature caused that? Shapley values come from cooperative game theory - the original question was "if a team wins a prize, how much did each player contribute?" The fair answer is to look at every possible ordering of players joining the team and average each player's marginal contribution. Apply that to features and you get: entropy contributed 41 points, extension changes 28, canary touches 19, and so on - and crucially **they sum exactly to the score**. That property is called efficiency, and it's why this isn't just feature importance hand-waving.

> SAY THIS
>
> *"Our explanations aren't a heuristic - they're exact. The per-feature contributions sum precisely to the anomaly score, which is a mathematical property of Shapley values called efficiency. So when the dashboard says entropy drove 40% of this alert, that's arithmetic, not a guess."*

3.5 Data leakage and why our splits are unusual

One line: Our windows overlap by 90%, so a random train/test split would cheat - we split by time and by session instead.

Plain English: We compute features over a 10-second window, but we slide it forward only 1 second at a time (so we can detect fast, instead of waiting for a full 10-second block). That means consecutive windows share 90% of the same file events. If you then shuffle all windows and split randomly, nearly every test window has an almost-identical twin in the training set. Your accuracy looks fantastic and means nothing. So we split **temporally** - train on earlier time, test on later - and we group by session so no attack episode appears on both sides. We also wrote a check, `assert_no_overlap`, that raises an error rather than letting a leaky experiment quietly produce a beautiful result.

> WHY THIS LANDS
>
> This is the answer that most impresses anyone who has actually reviewed ML papers. Volunteering a leakage risk that *you* found and *you* fixed is a much stronger signal than any accuracy figure - and there's a well-known USENIX Security paper (TESSERACT) making exactly this point about temporal splits in security ML. Cite it.

4. The live demo script, minute by minute

A demo that runs for eight minutes loses the room. This one is five, and every minute has a purpose. The rule: **never narrate what they can read on screen** - tell them what it *means*.

> **Before you walk in:** run the whole thing once, leave the terminal open, keep the dashboard tab loaded, and have `data/experiment_results.csv` already generated. If Wi-Fi or a fresh install can break it, it should already be done.

Minute 0:00-0:30 - The frame

Don't touch the keyboard yet. Deliver the 30-second pitch from Section 1. Let them hear the problem before they see any code.

Minute 0:30-1:30 - Normal life, then the attack

```
python demo.py
```

While phase 1 runs (learning normal behaviour):

> SAY THIS
>
> *"Right now it's just watching ordinary file activity - edits, saves, a backup copying files. This is the calibration phase, and it's the only data a real deployment can honestly collect: you can't ask a company to get ransomwared a hundred times so you can gather training data. That constraint is why we use anomaly detection rather than a supervised classifier."*

Then the attack phase starts. Point at the canary line the moment it appears.

> SAY THIS
>
> *"There. That's a canary file - a fake payroll spreadsheet that no legitimate program has any reason to open. Nothing should ever touch it. So the moment something does, we don't need statistics; that's a hostile act by definition, and the alert fires in under a second."*

Minute 1:30-2:30 - The explanation panel

Switch to the dashboard. Open one CRITICAL alert and read the evidence aloud.

> SAY THIS
>
> *"This is the part I'd ask you to judge us on. The alert doesn't just say 'anomaly, score 94'. It says a protected canary was touched, eighteen extensions changed in one window, peak entropy hit 7.9 when normal training never exceeded 5.2 - and the p-value is 0.0008, meaning under one in a thousand normal windows looks this strange. Every one of those numbers is a sentence a human can act on."*

> **WHY THIS LANDS**
>
> Explainability is where most student ML projects are weakest, so a clear evidence panel creates an outsized impression. You're not showing a model - you're showing a *decision someone could defend.*

Minute 2:30-3:30 - The partial-encryption ablation (your strongest slide)

Show the ablation table or the heatmap.

> **SAY THIS**
>
> *"Here's where it gets interesting. Real ransomware since about 2022 encrypts only a slice of each file - LockFile, BlackCat, Play, Qilin all do it - precisely because it defeats entropy detection. So we built that attack ourselves and turned the dial down to 15%.*
>
> *Entropy-only detection catches zero out of two. The full behavioural feature set catches two out of two. That's the whole argument of the project in one row: you cannot rely on entropy alone against a modern adversary."*

Minute 3:30-4:30 - Rollback with proof (the moment they remember)

```
python -m rr.respond.shadow
```

> **SAY THIS**
>
> *"Detection on its own is just a dashboard that tells you you've already lost. So before we escalate anything, we snapshot the original bytes. Watch: the file gets overwritten with random data - it's ruined - and then we restore it.*
>
> *And notice this line:* `verified_identical=True` *. That's not us printing a reassuring message. It re-reads every restored file, hashes it with SHA-256, and compares against the hash taken before the attack. The recovery is proved, not claimed."*

Minute 4:30-5:00 - The safety story (close on maturity)

```
python -m rr.respond.ladder
```

> **SAY THIS**
>
> *"Last thing. Once software can suspend and kill processes, it can do real damage, so most of this code is about refusing to act. Watch the refusals:* `lsass.exe` *is on a never-kill list, because killing it crashes Windows - that would be worse than the ransomware. Killing anything needs human confirmation. Everything is dry-run by default. There's a kill-switch file that disables every action instantly. And if we suspend a process wrongly, a timer resumes it automatically - a false positive heals itself.*
>
> *The audit log is hash-chained, so if anyone edits our history, verification points at the exact line that broke."*

WHY THIS LANDS

Ending on restraint rather than capability is a deliberate choice. Anyone senior in security will trust a student who says "here's what I refused to let it do" far more than one who says "it can kill any process".

5. The five things that make this stand out

If you remember nothing else, remember these five. Each one is something the typical project in your class will not have.

#	The differentiator	The one line that sells it
1	A metric that reflects reality: files lost vs false alarms per hour	"We optimise what a victim actually loses, not a leaderboard number."
2	Conformal p-values instead of hand-picked thresholds	"Our false-alarm rate is set in advance with a guarantee, not discovered later."
3	An adversary we built to defeat our own detector	"We attacked ourselves with the same evasion real families use."
4	Verified rollback, not just alerting	"The files come back, and SHA-256 proves they're identical."
5	A response system designed around refusing to act	"Dry-run by default, never-kill list, kill switch, self-healing false positives."

The bonus differentiator: a metric you deliberately deleted

There's a popular metric in anomaly-detection papers called point-adjusted F1. It was shown that a *random* scorer beats published state-of-the-art results under it - so a high score there proves nothing. Your code contains a function named `point_adjusted_f1_DO_NOT_USE` whose body raises an error explaining why.

> SAY THIS
>
> *"We could have reported point-adjusted F1 and shown a 0.97. We deliberately didn't, because a random scorer beats state-of-the-art under that metric - there's an AAAI paper on it. The function is still in our codebase, but it raises an exception explaining why you shouldn't call it. Choosing not to report a flattering number is also a result."*

> WHY THIS LANDS
>
> This is the single most memorable thing you can say in a viva. It demonstrates you read the literature, that you understand evaluation well enough to spot a broken metric, and that you had the discipline to walk away from an easy number. Very few undergraduate projects can claim that.

6. What to say to teachers and reviewers

Different people in the room want different things. Here's how to read them and what to lead with.

To the teacher who asks "where is the machine learning?"

This is the most likely challenge, and you have a very strong answer.

SAY THIS

"There are four ML components, and I'd argue the interesting one isn't the model at all. First, an unsupervised ensemble - ECOD plus Isolation Forest - over a 37-feature representation we engineered from raw filesystem events. Second, a conformal calibration layer that turns raw anomaly scores into p-values with a provable false-alarm bound. Third, Shapley attribution so every alert is explainable exactly, not approximately. Fourth, an evaluation protocol with leakage-safe temporal splits, bootstrap confidence intervals and McNemar tests for statistical significance between detectors.

The reason we use unsupervised learning isn't that it's easier - it's forced by the problem. You cannot collect a labelled dataset of a company being ransomwared. So you learn what normal looks like and measure deviation. That's the correct method for this domain, and it's what the published literature does too."

WHY THIS LANDS

Notice the structure: four components, then *why* the approach is forced by the domain. The second half is what turns "we used a library" into "we made a methodological choice". Deep learning would be the wrong answer here and you should say so if pushed: with a few thousand windows of self-generated data, a neural network would overfit and you'd have no calibration guarantee.

To the reviewer who asks "how is this different from antivirus?"

SAY THIS

"Antivirus mostly asks 'have I seen this file before?' - signatures. That fails by design against a brand-new variant. We never look at the malware at all. We look at behaviour: the pattern of file operations. A program that reads a thousand documents, writes them back as high-entropy data, renames them all and drops a text file called HOW_TO_DECRYPT is doing something recognisable regardless of what its binary looks like or whether anyone has seen it before."

To the practical person who asks "would this actually work?"

Answer honestly. The honesty is the impressive part.

SAY THIS

"Not as it stands, and I can tell you exactly why. Three reasons. One: `watchdog` gives us no process ID, so we can see what happened but not reliably who did it - a real deployment needs Sysmon or ETW. Two: our entropy check would false-positive on genuinely compressed files like real ZIPs and JPEGs; our sandbox is text-heavy, so a real deployment needs an extension allowlist. Three: all our numbers come from traffic we generated ourselves, so they show the method works, not that it generalises. The next step would be evaluating on a public trace dataset like RanSMAP."

WHY THIS LANDS

Pre-empting the three biggest holes yourself is disarming. Once you've named them, a reviewer has nothing left to catch you out with - and the conversation moves from "finding flaws" to "discussing the work", which is where you want it.

To the person who says "this looks like a lot for a minor project"

SAY THIS

"The scope grew because every time we measured something properly, it revealed a problem. We found our own data leakage, our own broken canary matching, and that our Isolation Forest was scoring 0.400 AUC while we thought it was working. Most of the work was fixing things measurement exposed, not adding features."

Three lines that consistently earn respect

1. **"We refused to report a metric that would have flattered us."** (Section 5)
2. **"We found a data leak in our own experiment and wrote a check that now fails loudly instead of quietly producing a nicer number."**
3. **"Our Isolation Forest scored 0.400 AUC - worse than random - and we left that in the README instead of hiding it."**

WHY THIS LANDS

All three are admissions. That is not an accident. Confident engineers report failures precisely; insecure ones hide them. Reviewers have seen hundreds of projects claiming 99% accuracy and know most of them are leaking data somewhere. Being the student who volunteers the ugly numbers reads as unusual maturity.

DO NOT SAY

Never say: "it's 100% accurate", "it detects all ransomware", "it's production-ready", or "we tested it on real ransomware" (you didn't, deliberately, and that's the right call - say *that* instead). Also avoid "we used AI" as a standalone claim; name the actual technique every time.

7. The viva Q&A bank

Answers written the way you'd actually say them out loud. If you internalise only ten, pick the starred ones.

1. Why anomaly detection instead of a classifier? ★

Because you can't get labels. Nobody can hand you a dataset of their company being encrypted a thousand times. Normal behaviour, though, is abundant. So we learn normal and measure distance from it.

2. What exactly is a feature window? ★

We group file events into 10-second and 60-second windows and compute 37 numbers per window - event rate, files touched, renames, extension changes, deletions, canary touches, peak entropy, plus smoothed trends and running totals. We slide the window forward 1 second at a time rather than in whole blocks, because a 10-second tumbling window puts a hard 10-second floor under your detection speed.

3. Why two window sizes?

A fast smash-and-grab attack shows up in 10 seconds. A deliberately slow attack looks normal over 10 seconds but abnormal over 60. Running both means we're not betting on the attacker's speed.

4. What is entropy, physically? ★

A measure of unpredictability in bytes, 0 to 8 bits. English text sits around 4.5 because letters are predictable. Encrypted or compressed data sits near 7.9 because every byte value is equally likely. A document jumping from 4.5 to 7.9 is a strong hint its contents were replaced with ciphertext.

5. Then why the chi-square test too?

Entropy can be fooled - you can construct data with high entropy that isn't uniform. Chi-square tests specifically whether all 256 byte values appear about equally often, which is what real encryption output looks like. Two independent views of the same 4KB sample is harder to evade than one.

6. How does a canary work, and why the realistic names? ★

They're decoy files - `Q4_Payroll_Report.docx`, `server_admin_passwords.txt` - that no legitimate program has any reason to open. Any access is hostile by definition, so the alert is instant and needs no statistics. The names are realistic because ransomware increasingly fingerprints and skips obvious decoys; a file called `canary_do_not_touch.txt` is a wasted tripwire.

7. What's your false positive rate?

With conformal calibration we *choose* it. At alpha = 0.01 we're bounded at 1% of benign windows by construction. We then report it in the units an operator cares about: false alarms per benign hour. We also require a k-of-n sequential agreement before alerting, which suppresses isolated spikes like a single backup job.

8. What happens on a legitimate backup or bulk file copy? ★

That's exactly the case that broke our first model - it flagged every backup. We fixed it by including bulk-copy activity in the calibration set, so "bulk is normal" becomes part of what the detector expects. Our benign workload generator includes seven such patterns: archive extraction, dependency installs, repo clones, photo imports, video exports, cloud sync and antivirus scans.

9. Why did your Isolation Forest score 0.400 AUC? ★

Because we trained it only on calm human activity, so anything fast looked anomalous - including legitimate backups - and it generalised badly to attacks. Worse than random, on that run. We kept the number in the README. The fix was broadening the calibration data and using a per-feature envelope for the actual decision, keeping the forest as a corroborating score rather than the judge.

10. How do you know your evaluation isn't leaking? ★

We assume it is until proved otherwise. Windows overlap by 90%, so we split temporally and group by session, and `assert_no_overlap` raises an exception if any test window shares time with training. We'd rather crash than publish an inflated number.

11. How long does detection take?

Canary touches alert in under a second because they don't need a window. Behavioural detection took about 6 seconds on average in our measurements. The honest framing is that latency is a dial, not a constant - and we report it against false alarms instead of quoting a best case.

12. Is the response system safe to run?

It's dry-run by default, so nothing happens unless you explicitly enable it. Beyond that: a never-kill allowlist for critical OS processes, killing requires human confirmation, there's a kill-switch file that disables everything, and reversible actions auto-revert on a timer so a false positive heals itself.

13. What if the ransomware kills your process first?

Then we lose, and we say so. That's a fundamental limit of user-space monitoring. The mitigation path is a kernel driver or a Windows service with tamper protection - real EDR products do this and it's out of scope for a minor project.

14. Could ransomware evade you deliberately?

Yes, and we built the tool to try. Our evader throttles its rate, encrypts only 15% of each file, avoids decoys, spreads across processes and randomises traversal order. Slow enough and it hides in the noise - so we report *where* that boundary sits instead of pretending it doesn't exist. Naming your own evasion limit is stronger than claiming you have none.

15. Why 37 features? Isn't that a lot?

They're grouped: raw counts, ratios, timing statistics, smoothed trends and cumulative totals. Ratios matter because absolute counts scale with machine speed but ratios don't. And we ran a leave-one-feature-out ablation, so we can tell you which ones actually earn their place rather than asserting they all do.

7b. Q&A bank, continued

16. What is the shadow journal, in one sentence? ★

A copy-on-write backup that saves a file's original bytes the first time something suspicious touches it, so we can put it back afterwards.

17. Why "first-write-wins"? ★

Because ransomware often writes the same file repeatedly. If we re-snapshotted on every write, the second snapshot would capture ciphertext and overwrite our only clean copy - our backup would destroy itself. So the first snapshot is permanent.

18. What if the journal fills up?

It refuses new snapshots rather than evicting old ones. Evicting would silently drop protection from a file we already promised to protect, and you'd only find out at restore time. Refusing is visible and honest.

19. What does the hash-chained audit log give you?

Tamper-evidence. Each line includes a hash of the previous line plus its own content, so editing or deleting any entry breaks every hash after it. `verify_audit_chain()` reports the exact line where the chain breaks. If this ever ran somewhere real, that log is the forensic record.

20. Why does quarantine only move newly-created files?

Because moving a pre-existing user file would itself be data loss - our response would become the damage. Only artefacts the attack created (ransom notes, `.locked` files) are eligible.

21. What's the TTL auto-revert for? ★

Self-healing false positives. If we suspend the wrong process, a timer resumes it and restores anything quarantined - no human needed. It means being wrong is recoverable, which is what lets us act at all.

22. Why is psutil not a dependency?

Process control sits behind an interface whose default implementation does nothing. That keeps the code importable and testable anywhere, and it means the dangerous capability is opt-in rather than always present.

23. How did you test something this stateful? ★

43 tests passing, and most of them test *refusals* rather than successes: escaping the root, killing an allowlisted process, exceeding the tier ceiling, skipping the human gate, tampering with the audit log. Process control is faked with an injected controller, and no test touches the network.

24. What's your biggest weakness?

The evaluation data is self-generated. Our simulator writes both the normal activity and the attacks, so we've shown the method is sound but not that it generalises to real machines. That's why public trace datasets are the top item on our future work, and why I'd frame these numbers as a method validation.

25. Did you run real ransomware?

No - deliberately, and I'd push back on any project that did in a college lab. Our attacker is our own reversible simulator, confined to a sandbox folder that must contain a sentinel file before it will run at all, and every change is XOR-reversible so the arena restores byte-for-byte.

26. What would you do with three more months?

Evaluate on RanSMAP and RanSAP public traces, switch the event source to the Windows USN journal for kernel-level fidelity, add Sysmon for real process attribution so response stops guessing, and map the rules to MITRE ATT&CK, starting with T1490, recovery inhibition.

8. If something breaks live

Something will. What separates a good demo from a bad one is entirely how you react. Never apologise twice, never debug silently, and never say "it worked yesterday".

What breaks	What you say, immediately	What you do
No desktop toast appears	"The toast is a PowerShell balloon and Windows Focus Assist eats it - the alert itself is in the dashboard, which is the source of truth."	Switch to the dashboard and keep going.
Dashboard won't load	"Streamlit's still binding the port - meanwhile the detector output in the terminal is the real signal, so let me walk you through that."	Narrate the terminal log; return to the browser later.
No alert fires on the attack	"That means the model hasn't been trained on this session yet - the calibration step is a prerequisite, which is itself the honest constraint of unsupervised detection."	Run `python model.py train`, then re-run the attack.
A false alarm fires during the benign phase	"And that's exactly the trade-off I was describing - at this alpha we accept roughly one percent false alarms. This is the cost side of the curve."	Turn it into the point. A live false positive that you predicted is credibility, not failure.
An import or dependency error	"Environment issue rather than a logic one - the test suite covers this module, so let me show you that instead."	Run `python -m pytest -q`. Green tests rescue almost any demo.
Total failure, nothing runs	"Let me show you the recorded numbers and the code instead - the experiment output is committed in `data/experiment_results.csv`."	Open the CSV and the README tables. Never debug in front of the panel for more than 30 seconds.

> WHY THIS LANDS
>
> The general principle: every failure has an explanation that makes you sound like you understand the system's limits. "Focus Assist suppresses balloon notifications" is a sentence that makes you sound more competent than a working toast would have.

Three sentences to keep in your pocket

- "That's a good question and it's a genuine limitation - here's how I'd fix it." *(Never bluff. Conceding well is stronger than defending badly.)*
- "I don't know, but I know how I'd find out." *(Then say how. This answer loses you nothing.)*
- "We measured that and the result surprised us." *(Then give the number. Nothing signals real work more than an unexpected result.)*

9. The cheat sheet

Print this page. Hold it. You won't need it, but holding it helps.

Open with

"Detection projects report accuracy. In ransomware, the only things that matter are how many files you lose before the alarm, and how often it cries wolf. We measure those two, we attack ourselves to prove it's real, and we roll the damage back with cryptographic proof."

The five numbers to know cold

Number	What it is
37	features per window (counts, ratios, timing, trends, totals)
10 s and 60 s	window sizes, slid forward 1 second at a time
7.9 / 8	entropy of encrypted content; normal text is ~4.5
15%	encryption fraction where entropy-only detection goes blind
0.400	our Isolation Forest's AUC on one run - worse than random, and we kept it in the README

Demo command order

```
python demo.py                 # phases 1-5: learn, train, attack, stealth, partial
python -m rr.respond.shadow    # rollback + verified_identical=True
python -m rr.respond.ladder    # five tiers, and every refusal
python -m pytest -q            # 43 passing, if anything goes wrong
```

The four sentences that do the most work

1. "A threshold is a number you guessed; a p-value comes with a guarantee."
2. "We built the attacker that defeats entropy detection, then showed our behavioural features still catch it."
3. "`verified_identical=True` means we re-hashed every restored file - the recovery is proved, not claimed."
4. "Most of the response code is about refusing to act: dry-run by default, never-kill list, human gate, kill switch, self-healing false positives."

Close with

"The honest summary: we've shown the method works on traffic we generated ourselves. We haven't shown it generalises to real machines - that needs public trace data, and it's the first thing we'd do next. But every number in here is one we measured, including the ones that made us look bad."

> **WHY THIS LANDS**
>
> Closing on a limitation is counter-intuitive and it works. It leaves the panel with the impression that you are the kind of person who tells the truth about your own work - which is, in the end, the only thing an examiner is really trying to assess.
